

Rapport de Travaux Pratiques : Algorithmes de Recherche de Chemin (Dijkstra & A*)

Réaliser par : **Mamadou NIMAGA**
Identifiant Étudiant : 22518134

Professeur : **Nicolas LOMENIE**

Date : 24/12/2025

Table des matières

1. Introduction	2
2. Modélisation et Structure de Données (Java)	2
2.1. Appropriation du Code de Base	2
2.2. Connectivité et Pondération	2
3. Algorithme de Dijkstra : Exploration Exhaustive	3
3.1 Pseudo-code de l'algorithme	3
4. Algorithme A* : Optimisation par Heuristique	4
4.1 Heuristique de Dijkstra Euclidienne	4
4.2 Pseudo-code de l'algorithme	4
4.3 Analyse Comparée des Performances	4
Tableau de comparaison : Dijkstra vs A*	5
Observation	5
5. Analyse de complexité Algorithmique	5
6. Réflexion sur les cas réels d'utilisation	5
6.1 Application de l'Algorithme de Dijkstra	5
6.2 Application de l'Algorithme A*	6
6.3 Applications en Finance Quantitative	6
7. Génie Logiciel et Documentation (Javadoc)	7
8. Conclusion Générale	7

1. Introduction

Ce rapport synthétise les travaux réalisés sur la recherche de chemin le plus court dans un graphe pondéré représentant une carte. L'objectif était de passer d'une approche de coloration (**Welsh-Powell**) vue précédemment en **C** à une approche de navigation intelligente en **Java**, en comparant l'algorithme de **Dijkstra** et l'algorithme de **A***.

2. Modélisation et Structure de Données (Java)

Contrairement aux TPs précédents utilisant des matrices d'adjacence en C, nous avons ici privilégié une approche orientée objet avec **des listes d'adjacence**.

2.1. Appropriation du Code de Base

Le projet s'appuie sur un squelette logiciel fourni (Sylvain Lobry) que j'ai dû compléter. Le code de base incluait :

- ★ La gestion de l'affichage graphique via la classe **Board**.
- ★ Le parsing des fichiers de données ('txt graph.txt').
- ★ La structure vide des classes 'Vertex' (Sommet) et 'Edge' (Arête).

2.2. Connectivité et Pondération

Mon travail a consisté à implémenter la création des arêtes pour établir un voisinage à **8 directions**.

- ★ **Poids horizontal/vertical** : Moyenne du coût de traversée ('indivTime') des deux sommets.
- ★ **Poids diagonal** : Application d'un facteur $\sqrt{2}$ au coût moyen pour respecter la distance euclidienne sur la grille.

```

// Ajout des arêtes
for (int line = 0; line < nlines; line++) {

    for (int col = 0; col < ncols; col++) {
        int source = line * ncols + col;
        int dest;
        double weight;

        for (int i = -1; i < 2; i++) {

            for (int j = -1; j < 2; j++) {
                // Si la case elle meme on continue
                if(i == 0 && j == 0) continue;

                int voisinLine = line + i;
                int voisinCol = col + j;
                if((voisinLine >= 0 && voisinLine < nlines) && (voisinCol >= 0 && voisinCol < ncols)) {
                    dest = voisinLine * ncols + voisinCol;
                    weight = (graph.vertexlist.get(source).indivTime + graph.vertexlist.get(dest).indivTime) / 2;
                    if(Math.abs(i) == 1 && Math.abs(j) == 1) {
                        weight *= Math.sqrt(2);
                    }

                    graph.addEgde(source, dest, weight);
                }
            }
        }
    }
}

```

Figure 1 : Code de la boucle imbriquée pour la création des arêtes avec `Math.sqrt(2)`

3. Algorithme de Dijkstra : Exploration Exhaustive

L'algorithme de Dijkstra explore les noeuds par coût croissant $g(n)$ depuis la source. J'ai implémenté la logique de relaxation des voisins en mettant à jour le champ `timeFromSource` et le parent `prev` pour la construction du chemin.

3.1 Pseudo-code de l'algorithme

L'algorithme suit cette logique fondamentale pour garantir le chemin le plus court :

1. **Initialisation** : Distances à l'infini, parent à nul, distance source à 0.
2. **Boucle principale** : Tant qu'il reste des sommets non visités :
 - 2.1. Pour chaque voisin v de u :

Si $distance(u) + poids(u, v) < distance(v)$:

Mettre à jour $distance(v)$ et définir u comme parent de v .

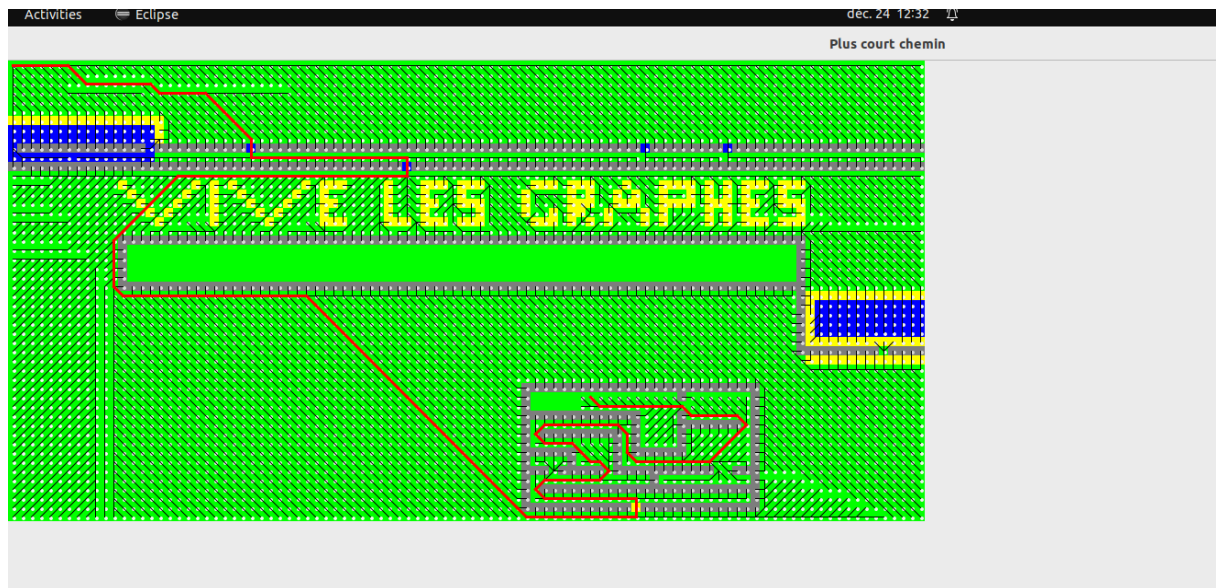


Figure 2 - Visualisation du chemin rouge tracé par Dijkstra sur la carte

4. Algorithme A* : Optimisation par Heuristique

4.1 Heuristique de Dijkstra Euclidienne

Pour A*, j'ai implémenté la méthode **estimation** calculant la distance à vol d'oiseau entre le sommet courant et la cible. Cette valeur $h(n)$ est ajoutée au coût réel $g(n)$ pour prioriser l'exploration vers l'objectif.

4.2 Pseudo-code de l'algorithme

A* utilise une fonction de priorité $f(n) = g(n) + h(n)$:

- $g(n)$: Coût réel depuis la source (identique à Dijkstra).
- $h(n)$: Estimation du coût vers la cible (Heuristique Euclidienne).
- **Logique** : A Chaque étape, on explore le noeud qui minimise $f(n)$, ce qui oriente la recherche vers la destination au lieu d'explorer tout le graphe.

4.3 Analyse Comparée des Performances

Grâce à l'ajout d'un menu interactif par **Scanner**, j'ai pu comparer les deux méthodes sur la même carte :

Tableau de comparaison : Dijkstra vs A*

Algorithme	Nœuds explorés	Temps total (Coût)
Dijkstra	4156	302.61
A*	4117	302.61

Observation

A* réduit le nombre de nœuds visités (**39** de moins sur ce test) tout en garantissant le même chemin optimal de **302.61**.

5. Analyse de complexité Algorithmique

Pour ces deux algorithmes appliqués à notre grille de **pixels**, la complexité est définie comme suit :

- ★ **Temporelle** : $O((m + n) * \log(n))$
 - $n = |V|$ représente le nombre de sommets (pixels).
 - $m = |E|$ représente le nombre d'arêtes (8-voisinage).
 - Le facteur $\log(n)$ provient de l'utilisation de **Priority Queue** pour l'extraction du minimum.
- ★ **Spatiale** : $O(n)$. Pour le stockage en mémoire les tableaux de distances et de parents pour chaque pixel.

6. Réflexion sur les cas réels d'utilisation

Mes expérimentations ont montré que si les deux algorithmes trouvent le même chemin optimal, A* est plus efficace car il explore moins de sommets grâce à l'heuristique. Cette différence permet d'identifier des cas d'utilisation distincts dans le monde réel :

6.1 Application de l'Algorithme de Dijkstra

L'algorithme de Dijkstra est privilégié lorsqu'on ne connaît pas la position géographique de la cible ou lorsqu'on doit calculer des chemins depuis une source vers **plusieurs** destinations simultanément :

- ➔ **Routage Réseau (Protocoles OSPF/IS-IS)** : Pour acheminer des paquets de données sur Internet via le chemin le plus rapide sans avoir de notion de "direction" géographique.

- **Système de Transport** : Pour calculer toutes les zones accessibles en moins de 30 minutes depuis un point précis (isochrones).
- **GPS (Calcul initial)** : Lorsque la destination est floue ou changeante, Dijkstra assure de trouver la solution brute la plus courte.

6.2 Application de l'Algorithme A*

A* est l'algorithme de choix lorsque la destination est fixe et que des coordonnées spatiales sont disponibles pour guider la recherche :

- **Jeux Vidéo** : C'est un algorithme standard pour le déplacement des personnages non-joueurs (PNJ). Il doit être extrêmement rapide pour ne pas ralentir le jeu, d'où l'importance de limiter les nœuds explorés.
- **Robotique et Drones** : Pour la navigation autonome d'un robot vers un point GPS précis. A* permet d'éviter les obstacles détectés en temps réel tout en fonçant vers la cible.
- **Applications de Cartographie (Google Maps / Waze)** : Pour calculer un itinéraire entre deux adresses précises, A* permet d'obtenir un résultat quasi instantané même sur des graphes de millions de nœuds.

6.3 Applications en Finance Quantitative

L'utilisation de Dijkstra et de ses variantes dépasse le cadre de la géographie physique pour s'appliquer à des "**espaces de prix**" abstraits :

- **Arbitrage de Devises (Triangulaire)** :
 - ◆ En finance, les taux de change entre devises peuvent être modélisés comme un graphe complet où chaque sommet est une devise (EUR, USD, JPY) et chaque arête est le taux de change.
 - ◆ Un algorithme de type Dijkstra peut être utilisé pour trouver le chemin de conversion le plus rentable.
 - ◆ *Note technique* : Puisque les taux de change se multiplient, les "**quants**" utilisent le logarithme négatif des taux ($- \ln(\text{taux})$) pour transformer le produit en une somme.
Trouver le "**chemin le plus court**" revient alors à trouver la séquence de transactions la plus lucrative.
- **Smart Order Routing (SOR)** :
 - ◆ Les grandes banques de finance de marché doivent exécuter des ordres d'achat/vente sur plusieurs places boursières (Euronext, Nasdaq, bourses privées) simultanément.
 - ◆ Chaque place a des frais de transaction et une liquidité différente (le "**poids**" de l'arête).
 - ◆ Dijkstra est utilisé pour router l'ordre via le chemin qui minimise le coût total d'exécution (frais + impact de marché).
- **Analyse de contagion et Risque Systémique**
 - ◆ On modélise le système financier comme un graphe où les sommets sont des banques et les arêtes sont des dettes croisées.

- ◆ Dijkstra permet de calculer la “**distance de défaut**” : à quel point la faillite d’une banque est “**proche**” d’en affecter une autre, permettant de mesurer la propagation d’une crise financière.

7. Génie Logiciel et Documentation (Javadoc)

Suivant une approche de Génie Logiciel, le projet a été versionné avec **Git**. Une attention particulière a été portée à la documentation :

- ★ Utilisation des tags **Javadoc** (`@param`, `@return`) pour chaque méthode.
- ★ Génération d’un site de documentation technique dans le dossier `/doc`.

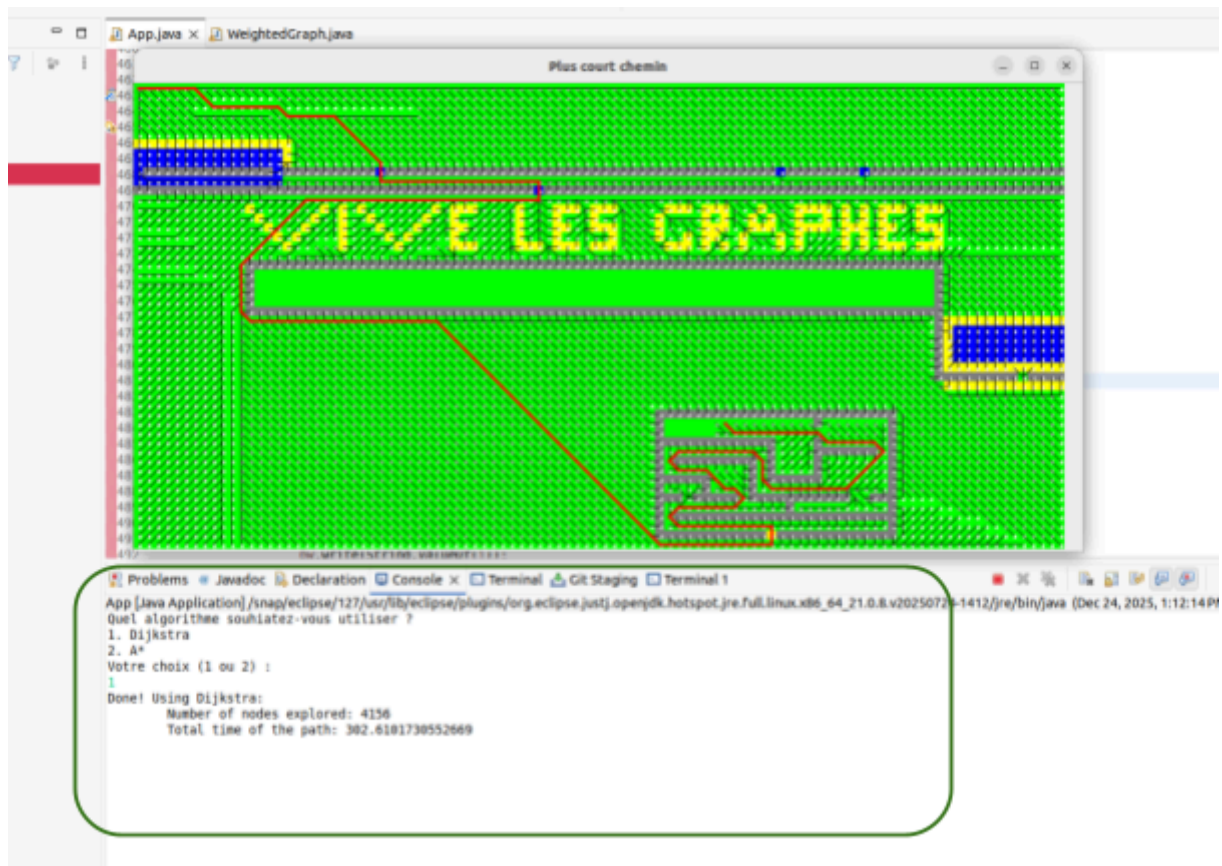


Figure 3 - Console Eclipse montrant le choix de l'algorithme et les résultats

8. Conclusion Générale

Ce TP a permis de consolider la manipulation des graphes pondérés en **Java**. L’implémentation de **A*** a démontré qu’une information supplémentaire (l’heuristique) peut optimiser significativement la recherche sans compromettre l’exactitude du résultat. Cette expérience souligne l’importance de choisir la bonne structure de données (liste d’adjacence) et le bon algorithme selon les contraintes de performance.